

PMI632/PM6150/PM7250B Qualcomm Battery Gauge PMIC Software

User Guide

80-PF208-74 Rev. G

April 11, 2022

For additional information or to submit technical questions, go to <https://createpoint.qti.qualcomm.com>

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer ("export") laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

Revision history

Revision	Date	Description
A	March 2018	Initial release
B	May 2018	Numerous changes have been made to this document. It should be read in its entirety.
C	February 2019	Updated for SM6125
D	September 2019	Editorial changes only
E	November 2019	Added Sections 2.21 through 2.23 and 4.3. Updated Sections 2.17 and 4.2.
F	July 2020	Added Section 2.24 and 2.25. Updated Section 2.16
G	April 2022	Updated parser.py information in Section 4.6

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-25 08:51:00 GMT
wangguanran@meigsmart.com

Contents

Revision history	2
1 Introduction	6
1.1 Purpose	6
1.2 Conventions	6
1.3 Technical assistance	6
2 Overview	7
2.1 Qualcomm Battery Gauge accumulator hardware	8
2.2 SoC estimation	8
2.3 Qualcomm Battery Gauge algorithm	8
2.4 Raw state of charge	9
2.5 Battery State of Charge	9
2.6 System available SoC	10
2.6.1 SoC point 0%	10
2.6.2 SoC point 100%	10
2.7 Monotonic SoC	10
2.8 PON SoC	11
2.9 Auto recharge SoC	11
2.10 Battery voltage low detection threshold	12
2.11 Empty SoC interrupt	12
2.12 Battery ESR estimation	13
2.13 Rconn	13
2.14 Delta SoC threshold	14
2.15 Hold SoC at 100% when the battery is full	14
2.16 Slope limit module parameters	14
2.17 Battery age detection	14
2.18 Charge cycle counter	15
2.19 BAT_ID pin	16
2.20 BAT_THERM pin	16
2.21 Fast charge config	16

2.22 Filtered voltage based SoC scaling	16
2.23 Multi-Profile loading based on age or SOH	16
2.24 Termination Current based SoC scaling	17
2.25 Battery SoC based SoC scaling	17
3 Qualcomm Battery Gauge driver	18
3.1 Driver SoC calculation	18
3.1.1 Kernel space	18
3.1.2 User space	19
3.2 INT handler	19
3.3 Power supply data structures	20
3.3.1 BMS power supply properties	20
3.3.2 Battery power supply properties	21
3.3.3 Power supply data structures	21
3.3.4 Power supply interfaces	22
3.4 Device tree	22
3.4.1 Device tree example	23
3.5 Battery profile	24
3.5.1 Battery characterization assistance	24
3.5.2 Example battery profile	24
4 Debug	25
4.1 Enable kernel space debug messages	25
4.2 Enable user space debug messages for QCT analysis	26
4.3 Enable user space debug messages for customer analysis	27
4.4 Execute UART/ADB shell to start logging	27
4.4.1 Enable capacity learning and cycle count logs	28
4.5 Get generated data file	28
4.6 Parse the generated data file	28
5 FAQ	29
5.1 Algorithm and measurement	29
5.1.1 How do I test the Qualcomm Battery Gauge performance?	29
5.1.2 Can the Qualcomm Battery Gauge SoC update cycle be changed?	29
5.1.3 Can the ESR measurements and/or pulses be disabled or their frequency changed?	29
5.1.4 What are the ~1 mA battery current pulses every 100 ms during device standby?	30
5.1.5 What if the apps wake up every 64 seconds during device active state?	30
5.1.6 What if the apps wake up every 64 seconds during device sleep?	31
5.1.7 The IBATT full termination current is inaccurate; what could be wrong?	31
5.2 Battery thermistor selection and measurement	31

5.2.1 The battery temperature measurements are out of specification; what could be wrong?	31
5.3 Battery identification resistor selection and algorithm	32
5.3.1 The battery RID accuracy is out of specification; what could be wrong?	32
5.3.2 What is the supported BATT_ID range and what happens if a value outside of that range is chosen?	32
5.4 Battery characterization	32
5.4.1 How do I submit batteries for characterization?	32
A References	33
A.1 Related documents	33
A.2 Acronyms and terms	33

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-25 08:51:00 GMT
wangguanran@meigsmart.com

1 Introduction

1.1 Purpose

This document describes Qualcomm Battery Gauge software driver configuration for PMI632 and PM6150 chipsets.

To configure the charging parameters in the device tree file, it is recommended to have a complete understanding of the hardware specifications described in *PMI632 Power Management Device Specification* (80-PF075-1).

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

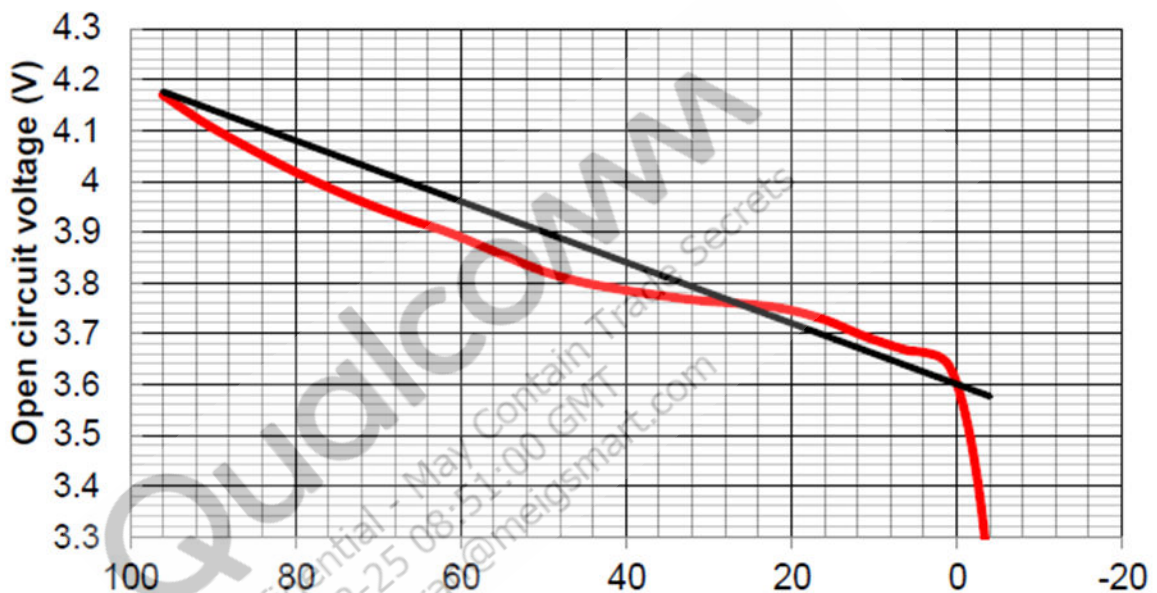
1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at <https://createpoint.qti.qualcomm.com/>.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

2 Overview

The Qualcomm Battery Gauge provides the necessary functions to calculate the state of charge (SoC) or remaining battery life based on an algorithmic approach. This software uses battery voltages, including open circuit voltage (OCV), if applicable.



The Qualcomm Battery Gauge is a combination of voltage and current-based SoC, providing the following features:

- Estimate of SoC is solely based on software
- Hardware that gives V_{batt} and I_{batt} values to software
- I_{batt} sensing through BATFET only
- Accurate end of charge (EoC) by charge termination based on measured I_{batt}
 - Digital detection of the termination current improves accuracy
- Online battery ESR estimation
- *Current Now* values for system power estimation
- Digital temperature detection for JEITA compliance
- Automatic recharge using SoC threshold

- Feature improvements/updates that can be done with OTA software updates
- Battery profile adopted from the fuel gauge for unified and accurate battery profiling (separate charge and discharge profiles at subzero temperatures)

For hardware block diagrams, refer to *PM8953 + PMI632 Power Management IC Design Guidelines/ Training Slides* (80-P2536-5C).

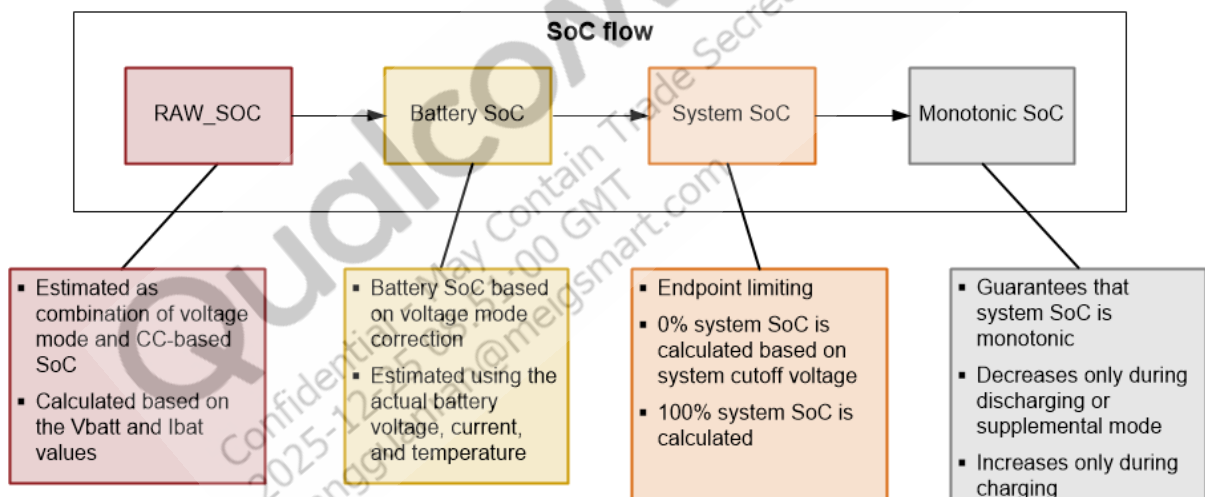
2.1 Qualcomm Battery Gauge accumulator hardware

The core of PMI632 Qualcomm Battery Gauge hardware consists of the accumulator, which periodically samples, accumulates, and saves Vbatt and Ibatt values into PMIC register to read and process SoC.

The same hardware is reused for an online battery ESR estimation, where V and I readings are run and saved to record pre-ESR and ESR pulse Vbatt and Ibatt values. The software reads the registers for ESR estimation.

For hardware block diagrams, refer to *PM8953 + PMI632 Power Management IC Design Guidelines/ Training Slides* (80-P2536-5C).

2.2 SoC estimation



2.3 Qualcomm Battery Gauge algorithm

The algorithm derives OCV from GOOD_OCV or FIFO Vbatt.

The following are considered GOOD_OCV:

- PON_OCV if device undergoes cold boot
- Shutdown SoC if device undergoes fast reboot
- Recorded OCV during sleep state

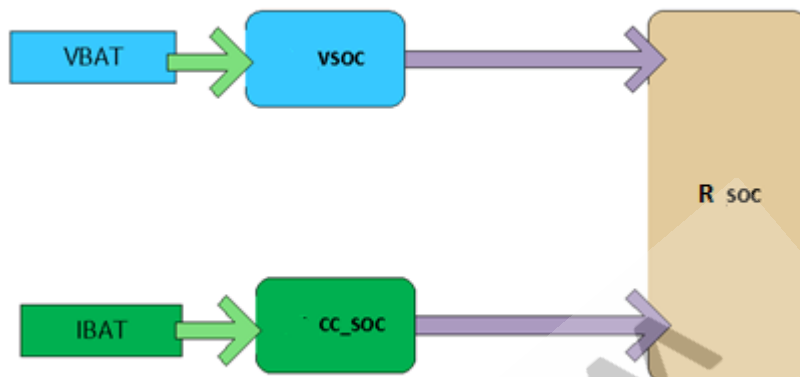
The calculated OCV is used to look up the corresponding SoC from the battery profile.

$$V_SOC = LUT(OVC, temperature)$$

The Qualcomm Battery Gauge algorithm derives CC_SOC based on FIFO Ibatt data.

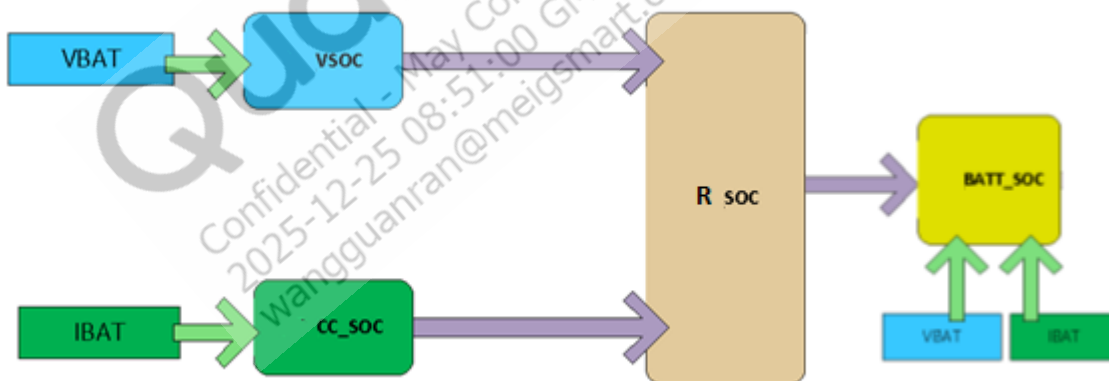
2.4 Raw state of charge

The Raw state of charge is a result of weighted average of Coulomb-Counted state of charge and Voltage mode state of charge.

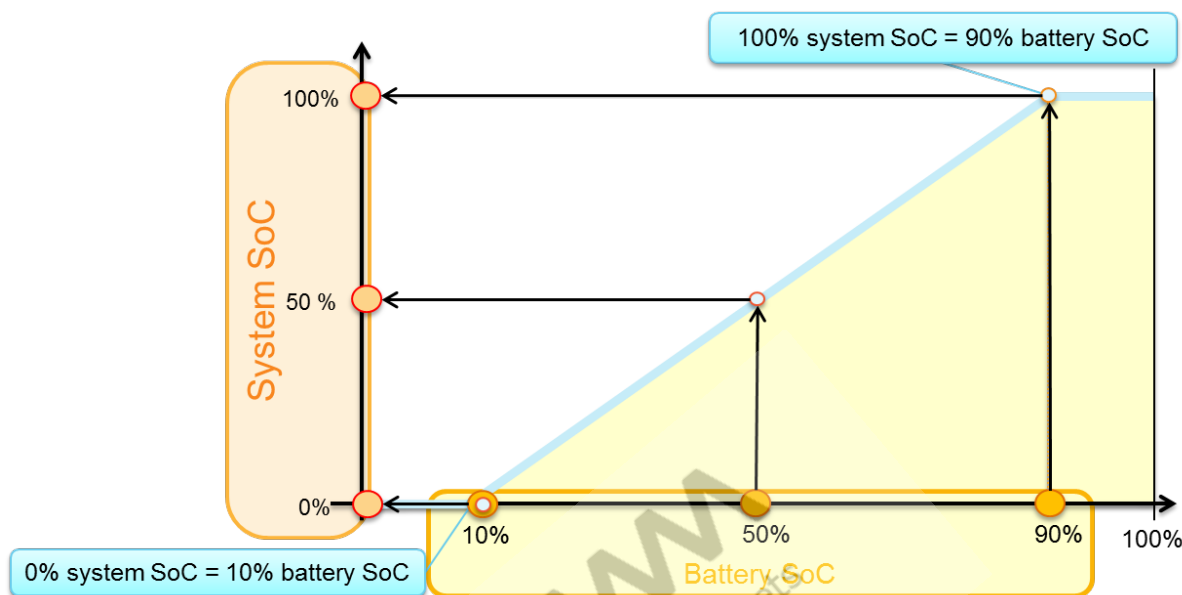


2.5 Battery State of Charge

Battery state of charge is estimated from the Raw state of charge and provides voltage loop feedback correction using battery voltage, current, and status values.



2.6 System available SoC



2.6.1 SoC point 0%

The reported 0% state of charge point is adaptively estimated based on cutoff voltage. The cutoff voltage setting is used to adjust the 0% state of charge point to any battery voltage desired.

```
gcom,vbatt-cutoff-mv = <3400>; /* example default 3.4V */
```

2.6.2 SoC point 100%

The 100% state of charge point is adaptively based on the programmable system termination current. This calculation predicts when 100% state of charge is displayed by using a programmable termination current threshold (system termination current).

The system termination current must be set at least 30 mA higher than the charger's end of charge termination current.

```
qcom,qg-iterm-ma = <130> ;      /* example 130 mA */
```

```
qcom,chg-term-current-ma = <100> ;    /* example 100 mA */
```

NOTE This property configuration must be defined as a charger-configuration parameter.

2.7 Monotonic SoC

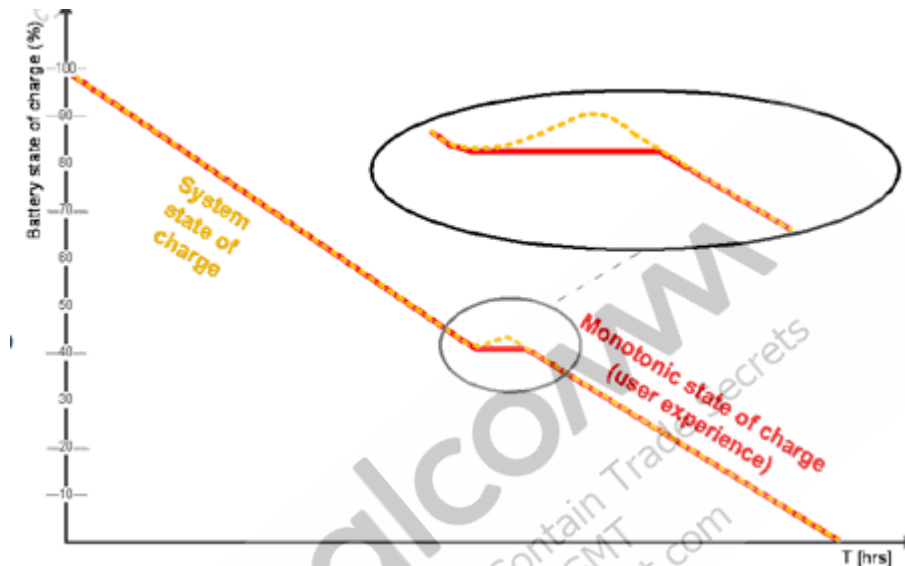
Because change in the operating conditions of the battery might result in an increase or decrease of the system available state of charge, value filtering is applied to obtain a desirable monotonic behavior.

Monotonic filtering handles situations in which the cutoff state of charge is increased and then decreased, which can happen in the following conditions:

- Increases and decreases in battery internal resistance value
- Increases and decreases in battery load current

Qualcomm Battery Gauge guarantees that the system state of charge is monotonic by maintaining the following:

- Decreases only during battery discharge or supplemental mode
- Increases only during battery charge



2.8 PON SoC

The PON SoC is typically derived from Shutdown SoC for phone restarts or when phone is in the off state for a short duration. For first-time boot or if phone is in the off state for a long time or if battery temperature varies a lot between shutdown and power on, Qualcomm Battery Gauge estimates the PON SoC based on the PON OCV reading taken during the phone boot. The conditions to ignore shutdown SoC and report PON SoC based on PON OCV can be controlled through following parameters:

```
qcom,ignore-shutdown-soc-secs = <360>; /* phone-off time to ignore shutdown soc */
```

```
qcom,shutdown-temp-diff = <60>; /* battery temp delta to ignore shutdown soc */
```

```
qcom,shutdown-soc-threshold = <10>; /*soc delta between shutdown and pon soc for ignoring shutdown soc */
```

2.9 Auto recharge SoC

Qualcomm Battery Gauge supports automatic recharge using an SoC-based threshold or voltage-based threshold. The default automatic recharge configuration is SoC-based threshold.

To recharge based on SoC, declare the following Boolean property in charger node:

```
qcom,auto-recharge-soc = <98>; /* default setting */
```

NOTE This property configuration must be defined as a charger-configuration parameter.

2.10 Battery voltage low detection threshold

This setting is the battery voltage threshold (in mV) at which the VBAT_LOW interrupt fires. Software performs more frequent SoC updates when this interrupt fires. The value of the `qcom,vbatt-low-mv` property represents the low battery voltage threshold in mV during normal temperature. This should be typically set 100 mV above the cutoff voltage.

```
qcom,vbatt-low-mv = <3500>; /* default setting */
```

The value of the `qcom,vbatt-low-cold-mv` property represents the low battery voltage threshold in mV during cold temperature. The threshold is only applied at the cold temperature specified by the `qcom,cold-temp-threshold` property. This should be typically set 400 mV above the cutoff voltage.

```
qcom,vbatt-low-cold-mv = <3800>; /* default setting */
```

The value of the `qcom,cold-temp-threshold` property represents the temperature threshold in decidegrees at which the low-temperature-specific configuration is as applied. If not specified, the default value is 0 degrees centigrade.

```
qcom,cold-temp-threshold = <0>; /* default setting */
```

NOTE Units are in decidegrees Celsius.

2.11 Empty SoC interrupt

The empty SoC interrupt is the battery voltage threshold (in mV) at which the vbatt-empty interrupt fires.

- Recommended setting/default = System cut-off voltage – 100 mV (might require more headroom than 100 mV value, depending on the system).
- Software forces a 0% upon an empty interrupt. User space is notified via the power supply framework. The user space reads 0% SoC and immediately shuts down to prevent UVLO.
- Configurable as device tree data (default value is 3200 mV).

```
qcom,vbatt-empty-mv = <3200>; /*default at normal temperature */
```

- The value of the `qcom,vbatt-empty-cold-mv` property represents the empty battery voltage threshold in mV during cold temperature. The threshold is only applied at cold temperature specified by the `qcom,cold-temp-threshold` parameter. This value should not be set below 3.0 V.

```
qcom,vbatt-empty-cold-mv = <3000>; /* default setting at cold temp */
```

2.12 Battery ESR estimation

The value of the equivalent series resistance (ESR) varies widely by the following factors:

- Temperature
- Battery residual capacity, that is, SoC
- Battery model

To guarantee an accurate SoC estimation, the algorithm must rely on a real estimate of the actual ESR. The Qualcomm Battery Gauge monitors the ESR variation by sampling valid synchronous readings of the battery voltage and current. The data collected is postprocessed to achieve the best estimation of the actual ESR. These pulses are generated every 32 minutes during charging by default and the entire procedure is run exceptionally, not actually affecting the battery lifetime.

Software reads this estimate via the `POWER_SUPPLY_PROP_RESISTANCE_NOW` power supply framework. The `POWER_SUPPLY_PROP_RESISTANCE` property value represents total `Rbatt` including ESR.

Table 2-1 ESR device tree properties

Parameter	Description
<code>qcom,esr-disable</code>	Boolean property to disable ESR estimation. If not defined, ESR estimation stays enabled during charging.
<code>qcom,esr-discharge-enable</code>	Boolean property to enable ESR estimation during discharge. Only valid if <code>qcom,esr-disable</code> is not defined.
<code>qcom,esr-qual-current-ua</code>	Minimum current differential in uA to qualify an ESR reading as valid. If not defined, the value defaults to 130 mA.
<code>qcom,esr-qual-vbatt-uv</code>	Minimum vbatt differential in uV to qualify an ESR reading as valid. If not defined, the value defaults to 7 mV.
<code>qcom,esr-disable-soc</code>	Minimum battery SoC below which ESR will not be attempted by Qualcomm Battery Gauge. If not defined, the value defaults to 10%.
<code>qcom,esr-chg-min-ibat-ua</code>	Minimum charge current (IBAT) in uA at which ESR will be attempted. If not specified, the default value is -450 mA.

2.13 Rconn

This property enables taking account of battery connector resistance (`Rconn`) to be used with ESR to get the `Rconn` in milliohms. This is retained for legacy reasons and should be set to zero.

```
qcom,rbat-conn-mohm= <0>; /* default setting */
```

2.14 Delta SoC threshold

This value specifies the delta change in SoC % at which SoC is reported to Android. If not specified, the value defaults to 1%. This means whenever the SoC changes by 1%, the SoC value is updated by the driver to Android.

```
qcom,delta-soc = <1>; /* default setting */
```

2.15 Hold SoC at 100% when the battery is full

If the charger is connected and SoC does not drop below the automatic recharge threshold, this feature shows the battery SoC at 100% when charging status reaches full.

Note: In some cases, such as JEITA conditions, the recharge SoC threshold is adjusted depending on the SoC at termination (full charge might be reached earlier than normal).

Use the following Boolean device tree property to enable this feature:

```
qcom,hold-soc-while-full;
```

Note: This property uses the qcom,auto-recharge-soc property as the threshold to resume charging.

qcom,linearize-soc is a Boolean property that when defined linearizes SoC when the SoC drops after charge termination monotonically to improve the user experience. This is applicable only if "qcom,hold-soc-while-full" is specified.

2.16 Slope limit module parameters

The slope limit feature defines the minimum time required to trigger the delta SoC interrupt to update the battery SoC. The parameters allow slope limit configuration based on the charging status and battery temperature threshold.

```
qcom,cold-temp-threshold
```

The battery temperature threshold determines what slope limit configuration must be applied with the charging status.

- 20 sec at normal temperature:

```
/sys/module/qg_soc/parameters/soc_interval_ms = 20000 /* default */
```

- 4 sec at cold temperature:

```
/sys/module/qg_soc/parameters/soc_cold_interval_ms = 4000 /* default */
```

2.17 Battery age detection

The battery learning capacity algorithm uses coulomb counting and the following information to discover battery capacity:

- Temperature
- Qualified SoC starting range
- Allowable increment and decrement of each charge cycle to qualify for each learning cycle
- Device tree parameter settings for battery age detection

Parameter	Description
<code>qcom,cl-disable</code>	Boolean property to disable the capacity learning.
<code>qcom,cl-feedback-on</code>	Boolean property to feedback the learned capacity into the capacity learning algorithm. This parameter cannot be used if the <code>qcom,cl-disable</code> property is specified.
<code>qcom,cl-max-temp</code>	Upper battery temperature limit to start the capacity learning.
<code>qcom,cl-min-temp</code>	Lower battery temperature limit to start the capacity learning.
<code>qcom,cl-max-increment</code>	Maximum percentage that the capacity can rise as the result of a single capacity learning cycle. This property corresponds to 0.1% increments.
<code>qcom,cl-max-decrement</code>	Maximum percentage that the capacity can fall as the result of a single capacity learning cycle. This property corresponds to 0.1% decrements.
<code>qcom,cl-max-limit</code>	Maximum limit that the capacity cannot go above in a capacity learning cycle.
<code>qcom,cl-min-limit</code>	Minimum limit that the capacity cannot go below in a capacity learning cycle.
<code>qcom,cl-max-start-soc</code>	Battery SoC must be below this value at the start of a charge cycle for capacity learning to be run.
<code>qcom,cl-min-start-soc</code>	Battery SoC must be above this value at the start of a charge cycle for capacity learning to be run.
<code>qcom,cl-wt-enable</code>	To enable weighted capacity learning based on change in battery SoC during a charging cycle. If this is specified, then <code>qcom,cl-min-start-soc</code> and <code>qcom,cl-max-start-soc</code> are not used.

2.18 Charge cycle counter

The charge cycle counters provide the number of times the battery is charged and are separated by bucket or charging cycle ranges. There are eight buckets allocated for the battery SoC (0 to 100%). For example, bucket ID 1 corresponds to charge cycle from 0 to 12.5%, ID 2 is 12.5 to 25%, and so on.

The bucket count is defined as follows:

```
#define BUCKET_COUNT 8
```

The bucket count for SoC is defined as follows:

```
#define BUCKET_SOC_PCT (256 / BUCKET_COUNT)
```

The bucket design increments the cycle counters based on multiple charging buckets. Because the user charges the battery at random levels, this design enables studying the charging pattern.

- The charge cycle counter is exposed through the following `cycle_count` psy property and it is an average of all the bucket counters:

```
POWER_SUPPLY_PROP_CYCLE_COUNT
```

- `CYCLE_COUNTS` property shows the bucket cycle counters as a string:

```
POWER_SUPPLY_PROP_CYCLE_COUNTS
```

2.19 BAT_ID pin

The Qualcomm Battery Gauge module is provided with a dedicated BAT_ID pin for battery missing detection and battery model identification.

The BAT_ID pin is accessible through the power supply framework (software read) as `POWER_SUPPLY_PROP_RESISTANCE_ID`.

2.20 BAT_THERM pin

The Qualcomm Battery Gauge module monitors battery temperature using a dedicated BATT_THERM pin via VADC. Information about the battery's temperature is used by the Qualcomm Battery Gauge for the following purposes:

- Improve SoC accuracy by adjusting the Qualcomm Battery Gauge models based on the temperature
- Accommodate charger operation with respect to JEITA requirements

Refer to *PMI632/PM6150/PM7250B Battery Temperature Sensing and Compensation Configuration* (80-P3255-39).

2.21 Fast charge config

If customers prefer to update the SoC during charging more frequently than 64 seconds, they can enable the following dtsti property and set the required frequency. The FIFO-Done IRQ is triggered more frequently based on the `fifo-length` set in the following dtsti property. For example, if the `fifo-length` value is 1, the SoC is updated every ~12 seconds during charging.

Parameter	Description
<code>qcom, qg-fast-chg-config</code>	Enables fast-charge configuration for QG. This allows configuring the FIFO length during fast charge.
<code>qcom, fast-chg-s2-fifo-length</code>	The FIFO length to be applied when system enters fast-charging. Takes effect only if <code>qcom, qg-fast-chg-config</code> is enabled. The default value, if not specified, is 1.

2.22 Filtered voltage based SoC scaling

If customers prefer to update the SoC based on filtered voltage, they can enable the following dtsti property. This logic enables SoC scaling to report 0% SoC at the cutoff voltage.

Parameter	Description
<code>qcom, fvss-enable</code>	Enables filtered voltage based SoC scaling.
<code>qcom, fvss-vbatt-mv</code>	Battery voltage threshold at which FVSS is enabled. Applicable only if <code>qcom, fvss-enable</code> is set.

2.23 Multi-Profile loading based on age or SOH

If customers prefer to load different profiles based on age or SOH of the battery, they must set the `qcom, multi-profile-load` dtsti property and assign an age/SOH level to each profile.

2.24 Termination Current based SoC scaling

If customers prefer to update the SoC based on termination current, they can enable the following dtsi property. This logic enables SoC scaling to report 100% SoC at the `qcom, qg-iterm-ma` current.

Parameter	Description
<code>qcom, tcss-enable</code>	Enables termination current based SoC scaling.
<code>qcom, tcss-entry-soc</code>	SOC threshold at which TCSS is enabled. Applicable only if <code>qcom, tcss-enable</code> is set.

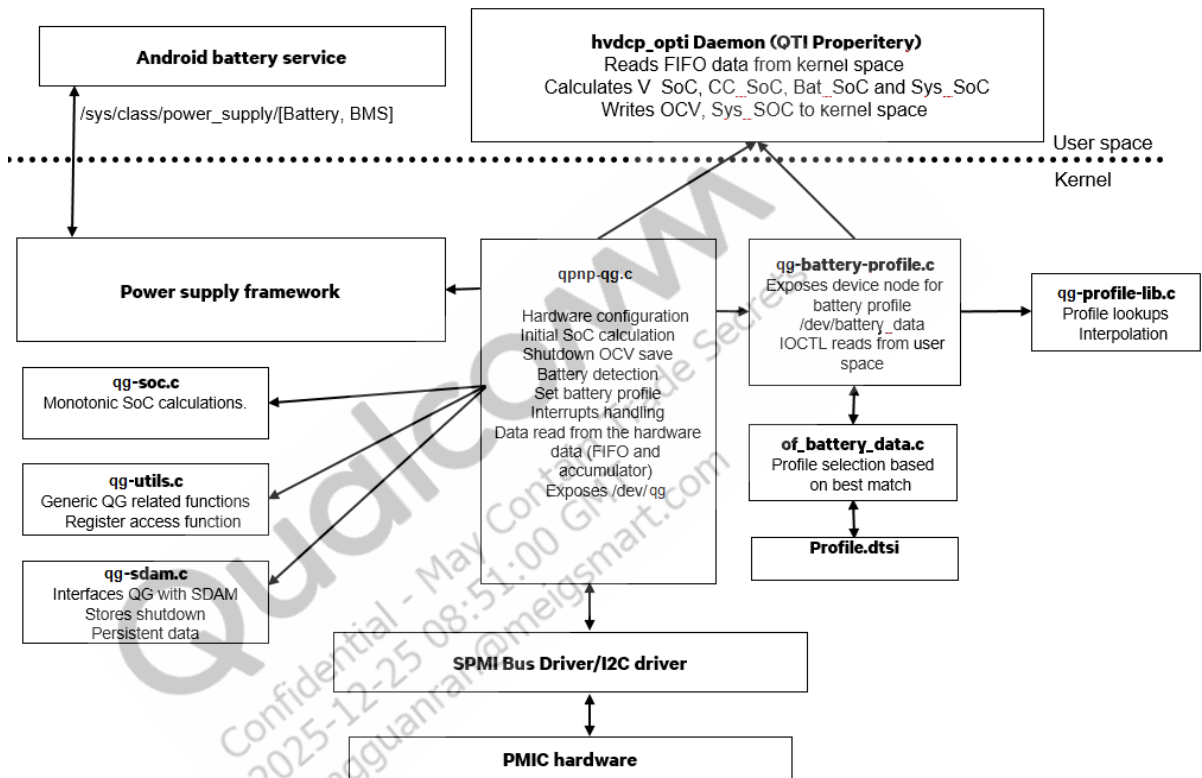
2.25 Battery SoC based SoC scaling

If customers prefer to update the Monotonic SoC based on Battery SoC, they can enable the following dtsi property. This logic allows monotonic-SOC scaling at low-temperatures when there is variation in system-SOC due to changes in the load.

Parameter	Description
<code>qcom, bass-enable</code>	Enable Battery SOC based SOC scaling.

3 Qualcomm Battery Gauge driver

The following diagram shows the Qualcomm Batter Gauge software architecture:



3.1 Driver SoC calculation

Qualcomm Battery Gauge SoC calculation is divided between kernel space and user space.

3.1.1 Kernel space

- Hardware configuration
- Initial SoC calculation
- Shutdown OCV save
- Battery detection
- Set battery profile

- Interrupt handling
- Data read from the hardware data (FIFO and accumulator)
- SoC lookup based on OCV (calculated in user space) and batt_temp
- Monotonic SoC calculations
- Driver source code is available in the following directory:
/drivers/power/supply/qcom

3.1.2 User space

- OCV calculation using hardware data (FIFO and accumulator) read from kernel space
- Algorithms calculating V_SoC, CC_SoC, R_SoC, Bat_SoC, and Sys_SoC
- Updates the calculated OCV, Sys_SOC to kernel space
- User space algorithm is shipped as a binary to the customers

3.2 INT handler

Qualcomm Battery Gauge SoC updates are triggered by hardware events which cover SoC updates required by software throughout all use cases or states.

The following table lists the PMIC interrupts supported by Qualcomm Battery Gauge.

Interrupt	Events
FIFO_UPDT_DONE	1. Notifies the driver about FIFO fill-up updates every 64 seconds by default during device active state, i.e., if the battery current is more than the <code>qcom,s3-entry-ibat-ua</code> default of 10 mA. 2. Reads FIFO and clears FIFO data. 3. Notifies the user space for SoC calculation. 4. Reads the user space data and updates the <code>QG_MON_SOC</code>.
GOOD_OCV	1. Notifies the driver about Good OCV update. 2. Notifies user space for SoC calculation. 3. Reads user space data and updates the SoC.
VBAT_LOW	1. Used in active or sleep states to notify software of a low battery condition if average Vbatt is less than the programmable thresholds. 2. FIFO_LEN is lowered further for frequent SoC updates.
VBAT_EMPTY	Used in active or sleep states to notify software of an empty battery condition if the average Vbatt is less than the programmable thresholds and shuts down the system gracefully.
BAT_MISSING or BAT_PRESENT	Debounced BATTERY_MISSING_INT from the Qualcomm Battery Gauge to notify software of a battery removal; if required, resets Qualcomm Battery Gauge data.

3.3 Power supply data structures

3.3.1 BMS power supply properties

Property	Description
POWER_SUPPLY_PROP_CAPACITY	Reported battery SoC. Value is a number between 0 to 100 representing the percent charge of the battery.
POWER_SUPPLY_PROP_VOLTAGE_NOW	Provides the instantaneous voltage of the battery.
POWER_SUPPLY_PROP_VOLTAGE_AVG	Provides the average voltage of the battery in the last 12.8 seconds.
POWER_SUPPLY_PROP_CURRENT_NOW	Provides the instantaneous current of the battery.
POWER_SUPPLY_PROP_RESISTANCE	Provides the estimated total battery resistance.
POWER_SUPPLY_PROP_RESISTANCE_NOW	Provides the profiled ESR battery resistance.
POWER_SUPPLY_PROP_RESISTANCE_CAPACITIVE	The battery capacitive resistance in milliohms. This is a legacy parameter and does not apply to the Qualcomm Battery Gauge.
POWER_SUPPLY_PROP_VOLTAGE_OCV	Describes the instantaneous OCV of the battery and is used to look up the current SoC of the battery. This is a writeable property and is periodically updated by the user space module.
POWER_SUPPLY_PROP_TEMP	Describes the current temperature of the battery. This is required by the Qualcomm Battery Gauge user space.
POWER_SUPPLY_PROP_BATTERY_TYPE	Describes the type of battery present in the system.
POWER_SUPPLY_PROP_CHARGE_FULL	Describes the actual FCC of the battery.
POWER_SUPPLY_PROP_CHARGE_FULL_DESIGN	Describes the nominal FCC of the battery.
POWER_SUPPLY_PROP_CYCLE_COUNT	Describes the number battery charging cycles.
POWER_SUPPLY_PROP_CHARGE_COUNTER	Provides the present charge count (CC) in uAH (CC_SOC*FCC).
POWER_SUPPLY_PROP_TIME_TO_FULL_AVG	Describes the estimated time required for battery to full.
POWER_SUPPLY_PROP_TIME_TO_EMPTY_AVG	Describes the estimated time required for battery to empty.
POWER_SUPPLY_PROP_SOH	Indicates the state of health (SOH) of the battery, which is calculated based on capacity learning and online ESR.
POWER_SUPPLY_PROP_ESR_ACTUAL	Describes the estimated ESR of the battery used in SOH estimation.
POWER_SUPPLY_PROP_ESR_NOMINAL	Describes the profiled ESR of the battery used in SOH estimation.
POWER_SUPPLY_PROP_VOLTAGE_MIN	Describes the VCutoff voltage used by Qualcomm Battery Gauge for converging to 0% SOC.
POWER_SUPPLY_PROP_VOLTAGE_MAX	Describes the Vfloat voltage used by Qualcomm Battery Gauge for calculating FULL_SOC.

3.3.2 Battery power supply properties

Property	Description
POWER_SUPPLY_PROP_STATUS	Describes the charge status of the battery: ■ POWER_SUPPLY_STATUS_CHARGING when the battery is charging ■ POWER_SUPPLY_STATUS_DISCHARGING when the battery is discharging ■ POWER_SUPPLY_STATUS_FULL when the battery is full

3.3.3 Power supply data structures

power_supply_propval

```
union    power_supply_propval
{
    int          intval;
    const char   *strval;
};
```

power_supply_type

```
enum     power_supply_type
{
    POWER_SUPPLY_TYPE_UNKNOWN = 0,
    POWER_SUPPLY_TYPE_BATTERY,
    POWER_SUPPLY_TYPE_UPS,
    POWER_SUPPLY_TYPE_MAINS,
    POWER_SUPPLY_TYPE_USB,
    POWER_SUPPLY_TYPE_USB_DCP,
    POWER_SUPPLY_TYPE_USB_CDP,
    POWER_SUPPLY_TYPE_USB_ACA,
    POWER_SUPPLY_TYPE_BMS,
};
```

power_supply_property

```
enum     power_supply_property
{
    POWER_SUPPLY_PROP_STATUS = 0,
    ...
    POWER_SUPPLY_PROP_ONLINE,
    ...
    POWER_SUPPLY_PROP_CURRENT_MAX,
    POWER_SUPPLY_PROP_CURRENT_NOW,
    ...
    POWER_SUPPLY_PROP_CHARGE_FULL_DESIGN,
```

```

...
POWER_SUPPLY_PROP_BATTERY_TYPE
POWER_SUPPLY_PROP_CAPACITY,
POWER_SUPPLY_PROP_VOLTAGE_OCV,
POWER_SUPPLY_PROP_TEMP,
POWER_SUPPLY_PROP_RESISTANCE,
POWER_SUPPLY_PROP_RESISTANCE_CAPACITIVE
...
};

```

3.3.4 Power supply interfaces

The power supply interface is in the following file:

```

kernel\include\linux\power_supply.h
struct    power_supply
{
    const char                *name;
    enum power_supply_type    type;
    enum power_supply_property *properties;
    ...
    int (*get_property)(struct power_supply *psy,
                        enum power_supply_property,
                        union power_supply_propval *val);
    int (*set_property)(struct power_supply *psy,
                        enum power_supply_property psp,
                        const union power_supply_propval *val);
    ...
    /* private */
    struct device              *dev;
    ...
};

```

3.4 Device tree

The following table describes DTSI properties configurable in the pmi632.dtsi.

kernel/arch/arm64/boot/dts/qcom/

Property	Description
qcom,vbatt-empty-mv	Battery voltage threshold (in mV) at which the vbatt-empty interrupt fires. The SoC is forced to 0 when this interrupt fires. If not specified, the default value is 3200 mV. This value is typically set 200 mV below the cutoff voltage.
qcom,vbatt-cutoff-mv	Battery voltage threshold (in mV) at which the Qualcomm Battery Gauge algorithm converges to 0 SoC. Default value is 3400 mV.
qcom,vbatt-low-mv	Battery voltage threshold (in mV) at which the VBAT_LOW interrupt fires. Software increases the SoC update rate for frequent SoC updates when this interrupt fires. Default value is 3500 mV. This value is typically set 100 mV above the cutoff voltage.

Property	Description
qcom,qg-iterm-ma	Battery current (in mA) at which the algorithm converges the SoC to 100% during charging and can be used to terminate charging. If not specified, the default value is 100 mA. This value must be at least 30 mA higher than the charger termination current so that SoC reaches 100% before charger termination.
qcom,delta-soc	This value specifies the delta charge in SoC % at which SoC is reported to Android. If not specified, the value defaults to 1%. This means that whenever the SoC changes by 1%, the SoC value is updated by the driver to Android.
qcom,s3-entry-fifo-length	This value specifies the consecutive number of FIFO current samples which are below the S3 IBAT entry threshold (qcom,s3-entry-ibat-ua) for Qualcomm Battery Gauge to enter into Sleep state. This value determines the time it takes for Qualcomm Battery Gauge hardware to transition from active to sleep state after the battery current has gone down below the sleep current threshold. Min Value = 1 Max Value = 8. The hardware default is configured to 3 (38.4 s). Total time taken to transition to sleep is (S3-entry-fifo-length*12.8 s).
qcom,s3-entry-ibat-ua	This value specifies the battery current threshold (in uA) for the Qualcomm Battery Gauge to enter the Sleep state. Qualcomm Battery Gauge hardware transitions into sleep state if battery current stays below this threshold consecutively for the FIFO length specified in 'qcom,s3-entry-fifo-length'. Min Value = 0 uA Max Value = 155550 uA Default Value = 10000 uA (10 mA), Step = 610 uA. This value should be set 2mA higher than the phone sleep current.
qcom,ignore-shutdown-soc-secs	This specifies the phone off time in seconds beyond which shutdown SoC is ignored for the subsequent PON SoC estimation. If not specified, the default value is 360 seconds.
qcom,shutdown-temp-diff	This specifies the allowed battery temperature delta in decidegree between shutdown battery temperature and poweron battery temperature to continue with the shutdown SoC as the PON SoC. If not specified, the default value is 6°C (60).
qcom,shutdown-soc-threshold	This value specifies the SoC delta allowed between PON and SHUTDOWN SoC for the shutdown SoC to be used as PON SoC. If the difference is beyond this value the PON SoC will be estimated based on OCV during PON.

3.4.1 Device tree example

```
pmi632_qg: qnp,qg {
    compatible = "qcom,qnp-qg";
    qcom,pmic-revid = <&pmi632_revid>;
    qcom,qg-vadc = <&pmi632_vadc>;
    qcom,vbatt-empty-mv = <3200>;
    qcom,vbatt-low-mv = <3500>;
    qcom,vbatt-cutoff-mv = <3400>;
    qcom,qg-iterm-ma = <100>;
    qcom,qgauge@4800 {
        status = "okay";
        reg = <0x4800 0x100>;
        interrupts = <0x2 0x48 0x0 IRQ_TYPE_EDGE_BOTH>,
        <0x2 0x48 0x1 IRQ_TYPE_EDGE_BOTH>,
        <0x2 0x48 0x2 IRQ_TYPE_EDGE_BOTH>,
        <0x2 0x48 0x4 IRQ_TYPE_EDGE_BOTH>,
        <0x2 0x48 0x5 IRQ_TYPE_EDGE_BOTH>,
        <0x2 0x48 0x6 IRQ_TYPE_EDGE_BOTH>;
        interrupt-names = "qg-batt-missing",
```

```

        "qg-vbat-low",
        "qg-vbat-empty",
        "qg-fifo-done",
        "qg-good-ocv",
        "qg-fsm-state-chg",
        "qg-event";
    };
    qcom,qg-sdam@b000 {
        status = "okay";
        reg = <0xb000 0x100>;
    };
};

```

3.5 Battery profile

To function properly, the Qualcomm Battery Gauge must have a specifically calibrated battery profile. A battery profile includes all the information necessary to allow the Qualcomm Battery Gauge to have the best possible SoC estimate.

3.5.1 Battery characterization assistance

File a Salesforce case with the following settings:

- Initial problem type – Hardware
- Case type – Tuning/optimization
- Problem area 1 – Hardware
- Problem area 2 – PMIC
- Problem area 3 – Battery profile

3.5.2 Example battery profile

The following file is an example battery profile:

```
kernel\msm-4.9\arch\arm64\boot\dts\qcom\qg-batterydata-ascent-3450mah.dtsi
```


4 Debug

4.1 Enable kernel space debug messages

For each of the following debug messages, `qg_debug_mask = 0x7FF`.

```
enum debug_mask {
    QG_DEBUG_PON          = BIT(0),
    QG_DEBUG_PROFILE      = BIT(1),
    QG_DEBUG_DEVICE       = BIT(2),
    QG_DEBUG_STATUS       = BIT(3),
    QG_DEBUG_FIFO         = BIT(4),
    QG_DEBUG_IRQ          = BIT(5),
    QG_DEBUG_SOC          = BIT(6),
    QG_DEBUG_PM           = BIT(7),
    QG_DEBUG_BUS_READ     = BIT(8),
    QG_DEBUG_BUS_WRITE    = BIT(9),
    QG_DEBUG_ALG_CL       = BIT(10),
};
```

Example log

```
6,11137,1171236239,-;QG-K: qg_device_write: QG write complete size=144
6,11138,1171236270,-;QG-K: process_udata_work: udata SOC=8 last SOC=9
6,11139,1171236275,-;QG-K: qg_scale_soc: SOC scale: Start msoc=9
catch_up_soc=8 delta_soc=1
6,11140,1171236360,-;QG-K: update_msoc: SOC scale: Update maint_soc=-22
msoc=8 catch_up_soc=8 delta_soc=1
6,11141,1171236367,-;QG-K: scale_soc_stop: SOC scale stopped: msoc=8
catch_up_soc=8
11,11142,1171236848,-;QG-D: Wrote 144 bytes to kernel
6,11143,1171240189,-;QG-K: qg_update_sdam_params: SDAM write param 0 value=1
6,11144,1171240198,-;QG-K: qg_update_sdam_params: SDAM write param 1 value=8
6,11145,1171240204,-;QG-K: qg_update_sdam_params: SDAM write param 2 value=277
6,11146,1171240210,-;QG-K: qg_update_sdam_params: SDAM write param 3 value=140
6,11147,1171240216,-;QG-K: qg_update_sdam_params: SDAM write param 4
value=3679900
6,11148,1171240222,-;QG-K: qg_update_sdam_params: SDAM write param 5 value=0
6,11149,1171240228,-;QG-K: qg_update_sdam_params: SDAM write param 6
value=1182
6,11150,1171240234,-;QG-K: qg_update_sdam_params: SDAM write param 7 value=0
```

```

6,11151,1171241272,-;QG-K: qg_status_change_work: charge_status=2
charge_done=0
6,11152,1171248061,-;QG-K: qg_get_learned_capacity: Retrieved learned
capacity 3481000uah
6,11153,1171256555,-;QG-K: qg_charge_full_update: msoc=8 health=1
charge_full=0
6,11220,1232609430,-;QG-K: qg_fifo_update_done_handler: IRQ triggered

```

4.2 Enable user space debug messages for QCT analysis

Due to internal sepolicy requirements– /dev/kmsg is allowable only for user/engineer-debug builds. Customers must modify the following file to enable logging in kmsg.

File: device/qcom/sepolicy/qva/vendor/common/hvdcpl.te

```

--userdebug_or_eng(`allow hvdcpl kmsg_device:chr_file rw_file_perms;')
++allow hvdcpl kmsg_device:chr_file rw_file_perms;

```

Going forward, customers must set the `qg.log_level` to 2 instead of 1 to get the entire Qgauge userspace logs. If `qg.log_level` is set to 1, only a few Qgauge parameters are logged, as shown in [Enable user space debug messages for customer analysis](#).

```

#adb root && adb wait-for-devices
#adb shell setprop persist.vendor.qg.log_level 2
#adb shell sync
#adb reboot

```

Example log

```

[ 71.852115] QG-D: FIFO=0 X1_DUMP [16610683438795] [0]
[ 71.852131] QG-D: FIFO=0 X2_DUMP [16610683438732] [0]
[ 71.852147] QG-D: FIFO=0 X3_DUMP [16610683438728] [0]
[ 71.852163] QG-D: FIFO=0 X4_DUMP [16610683438636] [1]
[ 71.852180] QG-D: FIFO=0 X5_DUMP [16610683012162] [0]
[ 71.852196] QG-D: FIFO=0 X6_DUMP [16610683421489] [0]
[ 71.852212] QG-D: FIFO=0 X7_DUMP [16610683438825] [0]
[ 71.852228] QG-D: FIFO=0 X8_DUMP [16610683438605] [0]
:
[ 71.852780] QG-D: FIFO=0 X42_DUMP [16610683440376] [0]
[ 71.852796] QG-D: FIFO=0 X43_DUMP [16610683452923] [0]
[ 71.852812] QG-D: FIFO=0 X44_DUMP [16610683438733] [0]
[ 71.852828] QG-D: FIFO=0 X45_DUMP [16610682271555] [0]
[ 71.852845] QG-D: FIFO=0 X46_DUMP [16610679082562] [0]
[ 71.852861] QG-D: FIFO=0 X47_DUMP [16610683439008] [0]
[ 71.852877] QG-D: FIFO=0 X48_DUMP [16610683438652] [0]

```

4.3 Enable user space debug messages for customer analysis

Customers can parse the Qgauge userspace logs themselves (see [Parse the generated data file](#)) and get the plot.xml file.

```
#adb root && adb wait-for-devices
#adb shell setprop persist.vendor.qg.log_level 1
#adb shell sync
#adb reboot
```

Example log:

```
11,24765,1981106315,-;QG-D: FIFO=0 TIMESTAMP_DUMP [633084]
11,24766,1981106322,-;QG-D: FIFO=0 SEQNO_DUMP [36]
11,24767,1981106327,-;QG-D: FIFO=0 FIFO_LENGTH_DUMP [5]
11,24768,1981106332,-;QG-D: FIFO=0 FIFO_V_DUMP [3723600]
11,24769,1981106336,-;QG-D: FIFO=0 FIFO_I_DUMP [-408630]
11,24770,1981106340,-;QG-D: FIFO=0 FIFO_CNT_DUMP [128]
11,24771,1981106344,-;QG-D: FIFO=0 B_TEMP_DUMP [284]
11,24772,1981106348,-;QG-D: FIFO=0 FCC_DUMP [4383]
11,24773,1981106352,-;QG-D: FIFO=0 INIT_OCV_DUMP [3685000]
11,24774,1981106356,-;QG-D: FIFO=0 GOOD_OCV_DUMP [0]
11,24775,1981106360,-;QG-D: FIFO=0 INIT_SOC_DUMP [1149]
11,24776,1981106364,-;QG-D: FIFO=0 CUTOFF_SOC_DUMP [205]
11,24777,1981106368,-;QG-D: FIFO=0 FULL_SOC_DUMP [9953]
11,24778,1981106371,-;QG-D: FIFO=0 CC_SOC_DUMP [975]
11,24779,1981106375,-;QG-D: FIFO=0 B_SOC_DUMP [966]
11,24780,1981106380,-;QG-D: FIFO=0 S_SOC_DUMP [780]
11,24781,1981106384,-;QG-D: FIFO=0 M_SOC_DUMP [800]
11,24782,1981106387,-;QG-D: FIFO=0 CC_UAH_DUMP [76225]
11,24783,1981106391,-;QG-D: FIFO=0 B_OCV_DUMP [3704800]
11,24784,1981106396,-;QG-D: FIFO=0 T_RBAT_DUMP [83]
11,24785,1981106400,-;QG-D: FIFO=0 ESR_DUMP [65]
11,24786,1981106404,-;QG-D: FIFO=0 ESR_CHG_SF_DUMP [6667]
11,24787,1981106408,-;QG-D: FIFO=0 ESR_DISCHG_SF_DUMP [6667]
```

4.4 Execute UART/ADB shell to start logging

Execute the following to start logging:

```
su
echo 10000 > /proc/sys/kernel/printk_ratelimit
echo 10000 > /proc/sys/kernel/printk_ratelimit_burst
echo "on" > /proc/sys/kernel/printk_devkmsg
echo 0 0 0 0 > /proc/sys/kernel/printk
cat /dev/kmsg | grep QG > /data/ kmsg.txt &
```

4.4.1 Enable capacity learning and cycle count logs

Enter the following command:

```
echo 'file fg-alg.c +p' > /sys/kernel/debug/dynamic_debug/control
cat /dev/kmsg | grep -E 'QG|ALG' > /data/ kmsg.txt &
```

Note: Ensure that there is a space preceding the ampersand (&) at the end of the command line.

4.5 Get generated data file

After completing the test, run the following commands to get the generated data file:

```
$ adb root
$ adb wait-for-devices
$ adb pull /data/kmsg.txt
```

4.6 Parse the generated data file

From the Attachments tab, download the TEMP file using the Readme instructions. Unzip the files to one location on your device and parse the Qgauge data with the following command:

```
parser.py kmsg.txt
```

The parser utility takes the plotting input from the plot xml file and generates a SoC plot Excel file (.xls).

5 FAQ

5.1 Algorithm and measurement

5.1.1 How do I test the Qualcomm Battery Gauge performance?

QTI does not support or provide any tools to customers for SoC accuracy testing. QTI recommends that customers develop their own knowledge and test methods for SoC accuracy testing. QTI tests the Qualcomm Battery Gauge extensively and has SoC accuracy data to provide over many use cases upon customer request.

If there are any concerns with Qualcomm Battery Gauge performance, QTI recommends that customers create a case, and QTI can review the logs during normal charge or discharge operation to evaluate its general performance.

5.1.2 Can the Qualcomm Battery Gauge SoC update cycle be changed?

A typical SoC update is based on the FIFO-done hardware interrupt every 64 seconds during the active state. If the estimated SoC jump is more than one percent, the software timer updates the SoC with a 20 second timer for normal temperatures and a 4 second timer for cold temperatures.

For example, if the current MSoC is 20% and catch_up_soc is 23% after the FIFO-done interrupt, a software timer of 20 seconds gradually increases the MSoC to 23%, instead of directly jumping to 23%.

5.1.3 Can the ESR measurements and/or pulses be disabled or their frequency changed?

By default the ESR measurements are enabled only during charging with a frequency of ~32 minutes.

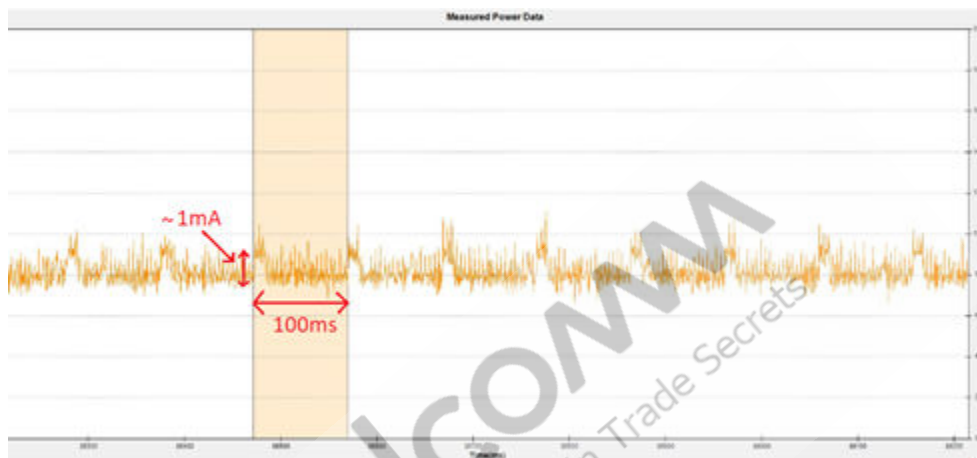
The ESR estimation can be disabled, but it is not recommended. It is strongly recommended that QTI be contacted before making the decision to disable ESR to verify that there will not be any unforeseen issues.

Although it is possible to change the ESR measurement interval timer, QTI strongly recommends against doing so and QTI does not guarantee the Qualcomm Battery Gauge SoC accuracy if the intervals have been changed, as any changes will differ from what has been validated internally.

5.1.4 What are the ~1 mA battery current pulses every 100 ms during device standby?

The Qualcomm Battery Gauge ADC measuring cycle is every 100 ms. During that time, the Qualcomm Battery Gauge measures voltage and current. These voltage and current measurements take ~20 ms, and cause the Qualcomm Battery Gauge module (ADC) to consume roughly 1 mA of current. See the following diagram for an example plot of these current pulses. These pulses last for ~36 seconds after the device enters sleep and then disappear until the next device wake up.

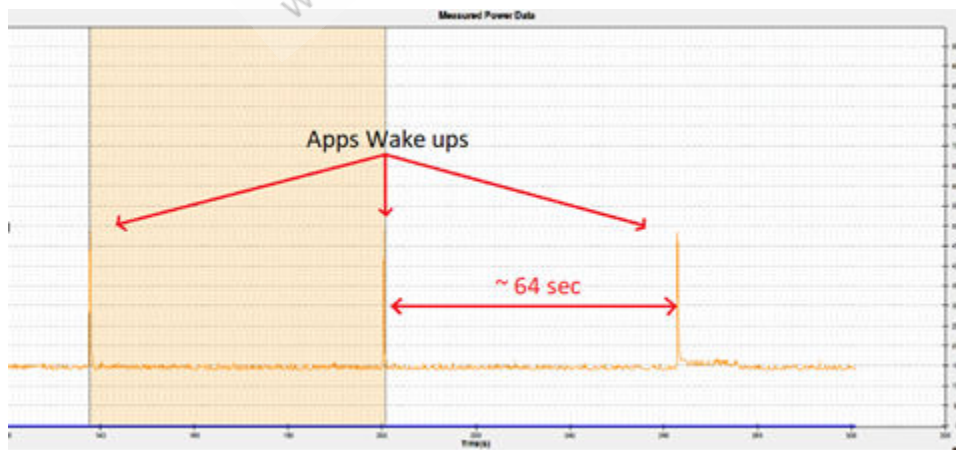
Once a device enters sleep, Qualcomm Battery Gauge is still in active state for 36 seconds and then it enters a sleep state, so the pulses will not be present after 36 seconds until the next wake up.



5.1.5 What if the apps wake up every 64 seconds during device active state?

Qualcomm Battery Gauge ADC measures voltage and current and fills the data into FIFOs. Once FIFOs are full, an interrupt to apps is triggered to process the data and estimate the SoC. These FIFOs take ~64 seconds to fill. The following diagram shows an example plot of these current pulses. These pulses every ~64 seconds during device static display use cases or voice calls.

Once the device enters sleep, Qualcomm Battery Gauge is still in an active state for 36 seconds and then enters a sleep state, so the pulses are not there if the device current consumption is less than `qcom, s3-entry-ibat-ua` (i.e., 10 mA).



5.1.6 What if the apps wake up every 64 seconds during device sleep?

Ideally, the device and apps should not wake up every 64 seconds if the device is in a sleep state (i.e., when the standby current is less than 10 mA; `qcom,s3-entry-ibat-ua`). If you observe that device is waking up every 64 seconds while in a sleep state, measure the standby current. If the current is above 10 mA, avoid the wake ups by increasing the sleep entry current threshold by modifying `qcom,s3-entry-ibat-ua` to the appropriate value.

For example, if the standby current is ~12 mA, then we can increase the threshold to ~18 mA.

```
qcom,s3-entry-ibat-ua = <18000> ; /* example 18 mA */
```

5.1.7 The IBATT full termination current is inaccurate; what could be wrong?

The feedback loop internal to the Qualcomm Battery Gauge responsible for matching the 100% point to the system termination current uses IBATT full settings. Make sure that online ESR estimation is enabled during charging or 100% SoC may not be reached. If poor accuracy is noticed, verify that these parameters are set correctly in the software. If these settings are incorrect, but accuracy is still poor, this could be a more serious issue, and it is recommended that a case be created via Salesforce to get help in debugging this further.

5.2 Battery thermistor selection and measurement

5.2.1 The battery temperature measurements are out of specification; what could be wrong?

Capacitance on BATT_THERM

This is a common issue that is attributed to added capacitance on the BATT_THERM net. Additional capacitance on this net is not recommended and may cause temperature measurement accuracy issues. QTI does not do internal validation with any capacitance on the BATT_THERM net. When checking for added capacitance, the battery pack's internal protection circuit is commonly overlooked, so be sure to check this as well.

Beta coefficients not programmed correctly

Another common issue is that the beta coefficients provided in [BAT_ID pin](#) are not programmed into the part. This causes the Qualcomm Battery Gauge temperature model to be shifted and read inaccurate temperature measurements. Verify that the correct coefficients are taken from the table and programmed in the software correctly.

Pull-up from BATT_THERM to BATT_THERM_BIAS not chosen correctly

The pull-up resistor from BATT_THERM_BIAS to BATT_THERM must match the thermistor's resistance value at 25°C. If these do not match, this affects the measurable temperature range and measurement accuracy.

5.3 Battery identification resistor selection and algorithm

5.3.1 The battery RID accuracy is out of specification; what could be wrong?

Battery RID accuracy being out of specification is a common issue that is attributed to added capacitance on the BATT_ID net. Additional capacitance on this net is not supported, and causes RID accuracy measurement issues. When checking for added capacitance, the battery pack's internal protection circuit is commonly overlooked.

5.3.2 What is the supported BATT_ID range and what happens if a value outside of that range is chosen?

Refer to the corresponding PMIC device specification for the supported BATT_ID ranges. If a BATT_ID value is chosen outside of QTI's supported ranges, BMD may mistrigger, and RID measurements cannot be guaranteed.

5.4 Battery characterization

5.4.1 How do I submit batteries for characterization?

Create a battery characterization case type (not a wireless device support). Fill in all necessary information, and send the batteries to the address in the case. If you have any questions, use the case to follow up with QTI.

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
<i>PMI632 Device Specification</i>	80-PF075-1
<i>PM632 Power Management IC Device Specification</i>	80-PF457-1
<i>Linux Android PMIC Software Drivers Overview</i>	80-PF208-23
<i>Linux Android PMIC Charger Software User Guide</i>	80-PF208-77
<i>PM8953 + PMI632 Power Management IC Design Guidelines/Training Slides</i>	80-P2536-5C
<i>PMI632 Battery Temperature Sensing and Compensation Configuration</i>	80-P3255-39

A.2 Acronyms and terms

Acronym or term	Definition
FCC	Full charge capacity – The total amount of charge that can be extracted from a fully charged battery. FCC changes with age and cycle life of the battery.
OCV	Open circuit voltage – Battery voltage at a steady state near-zero current (generally less than C/20). Voltage takes 5-30 min to settle to OCV after a load.
Rbatt	Battery resistance – Internal resistance of the battery.
SoC	State of charge – Ratio of the remaining capacity to the total capacity.